แบบฟอร์มการเสนอหัวข้อ (Final Project Form)

ส่วนที่ 1: ข้อมูลพื้นฐาน (Basic Information)

- ชื่อโครงงาน (Project Title):
    - LogSentinel: ระบบตรวจจับความผิดปกติใน System Log ด้วย Deep Learning สำหรับศูนย์ปฏิบัติการความปลอดภัยไซเบอร์ (SOC)
    - *LogSentinel: Deep Learning-based Log Anomaly Detection for Security Operations Centers*
- ชื่อ-สกุล และรหัสนักศึกษา (Student Name & ID):
    - ธนกร จำนงประโคน 1660701168
    - ศิวกร สุภาวงค์ 1660700251
    - นิธิสิทธิ์ เสนาหาญ 1660701317
    - ภูวดล ผิวบางกูล 1660702067

ส่วนที่ 2: ภาพรวมโครงงาน (Project Overview)

1. บทคัดย่อ (Abstract)

ระบบคอมพิวเตอร์ในองค์กรสร้าง log จำนวนมหาศาลทุกวัน จนนักวิเคราะห์ใน SOC ตรวจสอบได้ไม่ทั่วถึง และระบบแจ้งเตือนแบบ rule-based มักตรวจไม่พบความผิดปกติรูปแบบใหม่ โครงงานนี้จึงนำ Deep Learning มาตรวจจับความผิดปกติจาก log โดยอัตโนมัติ โดยแปลง log ดิบเป็นลำดับเหตุการณ์ด้วย Drain แล้วใช้โมเดล LSTM และ Transformer เรียนรู้รูปแบบการ

ทำงานปกติของระบบ เพื่อระบุว่า session ใดผิดปกติ ทดลองกับชุดข้อมูล HDFS และ BGL จาก Loghub และเปรียบเทียบกับ baseline ทาง Machine Learning โดยคาดหวังค่า F1-Score ไม่ต่ำกว่า 80% พร้อมเว็บแอปต้นแบบที่ให้ผู้ใช้อัปโหลดไฟล์ log แล้วแสดง session ที่ผิดปกติและเหตุการณ์ที่น่าสงสัยได้

2. ที่มาและความสำคัญของปัญหา (Problem Statement)

- **ปัญหาคืออะไร (What is the problem?):**
  - ปริมาณ log มีมากเกินกว่าที่คนจะอ่านได้ทัน เช่น ชุดข้อมูล HDFS เพียงชุดเดียวมีมากกว่า 11 ล้านบรรทัดภายในเวลาไม่ถึง 2 วัน
  - เหตุการณ์ผิดปกติมีสัดส่วนน้อยมาก (ประมาณ 2.9% ของ session ใน HDFS) ทำให้ถูกกลบอยู่ใน log ปกติจำนวนมหาศาล
  - การตรวจจับด้วย rule หรือ regex ต้องเขียนและดูแลด้วยมือ ตามไม่ทันรูปแบบความผิดปกติหรือการโจมตีใหม่ ๆ และมักสร้าง false alarm จำนวนมากจนเกิดภาวะ alert fatigue
- **เกิดขึ้นในบริบทใด (Business Domain):**
  ความปลอดภัยไซเบอร์และการปฏิบัติการด้านไอที (Cybersecurity / IT Operations) ได้แก่ งาน SOC, SIEM, Incident Response และ AIOps
- **ใครคือผู้ที่ได้รับผลกระทบ หรือจะได้รับประโยชน์จากโซลูชันนี้ (Who benefits?):**
  - นักวิเคราะห์ SOC (โดยเฉพาะ Tier 1) ที่ต้องคัดกรอง log และ alert จำนวนมากในแต่ละวัน
  - ผู้ดูแลระบบ, ทีม DevOps/SRE และทีม Incident Response ที่ต้องหาสาเหตุเมื่อระบบขัดข้องหรือถูกโจมตี

  - องค์กรขนาดกลางและเล็กที่มีบุคลากรด้านความปลอดภัยจำกัด และไม่สามารถเขียน rule ครอบคลุมทุกกรณีได้

- **การแก้ปัญหานี้จะสร้างผลกระทบ (Impact) อย่างไร?:**
  - ลดเวลาในการตรวจพบเหตุผิดปกติ (Mean Time to Detect: MTTD) และลดภาระการไล่อ่าน log ด้วยมือ
  - ตรวจพบรูปแบบความผิดปกติที่ไม่เคยมี rule รองรับ เพราะโมเดลเรียนรู้จาก "พฤติกรรมปกติ" แทนการจับคู่ signature
  - ชี้จุดเหตุการณ์ที่น่าสงสัยใน log sequence ช่วยให้การสืบสวน (investigation) เร็วขึ้น และใช้เป็นส่วนเสริมของ SIEM ที่มีอยู่ได้

3. วัตถุประสงค์ของโครงงาน (Project Objectives)

- เพื่อพัฒนาโมเดล Deep Learning สำหรับ Log Anomaly Detection (Binary Classification ระดับ session: Normal / Anomaly) บนชุดข้อมูล HDFS ด้วยค่า F1-Score ไม่ต่ำกว่า 80%
- เพื่อสร้างแอปพลิเคชันต้นแบบ (Prototype) ด้วย Streamlit ที่ให้ผู้ใช้อัปโหลดไฟล์ log (.log / .csv) หรือวางข้อความ log แล้วแสดงผลการทำนายว่าแต่ละ session ปกติหรือผิดปกติ พร้อม anomaly score และไฮไลต์เหตุการณ์ที่น่าสงสัย
- เพื่อเปรียบเทียบประสิทธิภาพของโมเดล LSTM (แนวทาง DeepLog) และ Transformer Encoder กับ baseline ทาง Machine Learning (Isolation Forest และ Logistic Regression) รวมถึงทดสอบความสามารถในการใช้งานกับ log ต่างระบบบนชุดข้อมูล BGL

ส่วนที่ 3: รายละเอียดทางเทคนิค (Technical Details)

1. ชุดข้อมูลที่ใช้ (Dataset)

- ชื่อ Dataset: HDFS_v1 (ชุดข้อมูลหลัก) และ BGL (ชุดข้อมูลเสริม) จากคลังข้อมูล Loghub
- แหล่งที่มา (Link to Dataset): – https://github.com/logpai/loghub
  – https://zenodo.org/records/8196385
- รายละเอียดเบื้องต้น: – HDFS_v1: log จากระบบ Hadoop Distributed File System จำนวน 11,175,629 บรรทัด ครอบคลุมช่วงเวลา 38.7 ชั่วโมง (ขนาด 1.47 GiB) จัดกลุ่มตาม block_id ได้ประมาณ 575,061 session มี label 2 คลาส (Normal / Anomaly) โดยเป็น Anomaly ประมาณ 16,838 session (≈2.9%)

  – BGL: log จาก supercomputer BlueGene/L จำนวน 4,747,963 บรรทัด ครอบคลุม 214.7 วัน (ขนาด 708.76 MiB) มี label ระดับบรรทัด (alert / non-alert) ใช้ทดสอบกับ log ต่างประเภท

  - รูปแบบไฟล์: ไฟล์ข้อความ log ดิบ (.log) และไฟล์ label (.csv)

2. แนวทางการดำเนินงาน (Proposed Methodology)

- สถาปัตยกรรมโมเดลที่เลือกใช้ (Model Architecture):
  – **LSTM (แนวทาง DeepLog):** เรียนรู้จากลำดับเหตุการณ์ปกติเพื่อทำนายเหตุการณ์ถัดไป หากเหตุการณ์จริงไม่อยู่ใน top-k ที่ทำนายจะถือว่าผิดปกติ เหมาะเพราะ log เป็นข้อมูลลำดับเวลา และไม่ต้องใช้ label ของความผิดปกติมาก ซึ่งตรงกับสถานการณ์จริงที่ข้อมูลการโจมตีหายาก

  – **Transformer Encoder:** ใช้ self-attention จับความสัมพันธ์ของเหตุการณ์ที่อยู่ห่างกันในลำดับได้ดีกว่า RNN ประมวลผลแบบขนานได้เร็ว และใช้ค่า attention ช่วยอธิบายว่าเหตุการณ์ใดทำให้ session ถูกตัดสินว่าผิดปกติ

  – **Baseline:** Isolation Forest และ Logistic Regression บน event count vector เพื่อเป็นเกณฑ์เปรียบเทียบว่า Deep Learning ให้ผลดีกว่าวิธีดั้งเดิมจริงหรือไม่

- เครื่องมือและไลบรารี (Tools & Libraries): Python, PyTorch, Drain3 (log parsing), Pandas, NumPy, Scikit-learn, Matplotlib / Seaborn, Streamlit (prototype), Google Colab / GPU, Git และ GitHub

3. เกณฑ์การวัดผล (Evaluation Plan)

- Metric หลักที่ใช้วัดผล: F1-Score (หลัก), Precision, Recall, PR-AUC และ Confusion Matrix
  เหตุผลที่เลือกใช้ Metric นี้:
  - ชุดข้อมูลไม่สมดุลอย่างมาก (Anomaly ≈2.9%) หากใช้ Accuracy โมเดลที่ทายว่า "ปกติ" ทุก session ก็ได้ความแม่นยำถึงประมาณ 97% จึงไม่สะท้อนความสามารถจริง
  - Recall สำคัญเพราะการพลาดความผิดปกติ (False Negative) หมายถึงการพลาด incident จริง ส่วน Precision สำคัญเพื่อลด false alarm ที่ทำให้นักวิเคราะห์ SOC เกิด alert fatigue
  - F1-Score สมดุลทั้งสองด้าน และเป็น metric มาตรฐานในงานวิจัยด้านนี้ (เช่น DeepLog, LogBERT) ทำให้เทียบผลกับงานที่มีอยู่ได้ ส่วน PR-AUC ใช้ประเมินโมเดลโดยไม่ขึ้นกับ threshold

ส่วนที่ 4: แผนการดำเนินงานและผลลัพธ์ (Plan & Deliverables)

1. แผนการดำเนินงาน (Timeline)

สัปดาห์	งานที่ดำเนินการ
Week 1	ศึกษางานวิจัยที่เกี่ยวข้อง (DeepLog, LogBERT), ดาวน์โหลดชุดข้อมูล HDFS / BGL และทำ EDA
Week 2	Log parsing ด้วย Drain, สร้าง session / log sequence และแบ่งข้อมูล train / validation / test
Week 3	สร้างโมเดล baseline (Isolation Forest, Logistic Regression) และ pipeline สำหรับวัดผล

สัปดาห์	งานที่ดำเนินการ
Week 4	พัฒนาและฝึกโมเดล LSTM (แนวทาง DeepLog) พร้อมปรับ hyperparameter
Week 5	พัฒนาและฝึกโมเดล Transformer Encoder พร้อมปรับ hyperparameter
Week 6	เปรียบเทียบผลทุกโมเดล, ทดสอบบน BGL และวิเคราะห์ข้อผิดพลาด (error analysis)
Week 7	พัฒนาเว็บแอปต้นแบบด้วย Streamlit และเชื่อมต่อกับโมเดลที่ดีที่สุด
Week 8	ทดสอบระบบ, จัดทำรายงาน, สไลด์นำเสนอ และ demo

ผลลัพธ์ที่ส่งมอบ (Deliverables): Jupyter Notebook สำหรับเตรียมข้อมูลและฝึกโมเดล, โมเดลที่ผ่านการฝึก, เว็บแอปต้นแบบ (Streamlit), รายงานผลการเปรียบเทียบโมเดล และ source code บน GitHub

ส่วนที่ 5: เอกสารอ้างอิง (References)

[1] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. ACM CCS 2017. https://doi.org/10.1145/3133956.3134015

[2] Zhu, J., He, S., He, P., Liu, J., & Lyu, M. R. (2023). Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. IEEE ISSRE 2023. https://arxiv.org/abs/2008.06448

[3] Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log Anomaly Detection via BERT. IJCNN 2021. https://arxiv.org/abs/2103.04475

[4] He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An Online Log Parsing Approach with Fixed Depth Tree. IEEE ICWS 2017.

[5] Xu, W., Huang, L., Fox, A., Patterson, D., & Jordan, M. I. (2009). Detecting Large-Scale System Problems by Mining Console Logs. ACM SOSP 2009.

[6] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS 2017. https://arxiv.org/abs/1706.03762

[7] Loghub (GitHub): https://github.com/logpai/loghub

[8] Loglizer – Machine learning-based log analysis toolkit (GitHub): https://github.com/logpai/loglizer

[9] Drain3 (PyPI): https://pypi.org/project/drain3/